ENAE464 - 0202

Lab06: Transonic Flow in a Quasi-1D Nozzle

Due on May 8th, 2026 at 11:59 PM

Dr. Silbaugh, 02:00 PM

Mikołaj Kostrzewa & Vai Srivastava

Report Submitted: May 8th, 2026

May 8$^{\text{th}}$, 2026

Enclosed is the technical report for the *Transonic Flow in a Quasi-1D Nozzle* computational experiment. This report presents the computational methodology, results, and analysis of flow characteristics and effects for a quasi-1D nozzle subjected to transonic flow.
Please feel free to contact us with any questions regarding the contents of this report.

Respectfully,
Mikołaj Kostrzewa & Vai Srivastava

Contents

Figures

Tables

Listings

1 Abstract

The transonic flow in a quasi-one-dimensional diverging nozzle was investigated using two complementary computational fluid dynamics techniques implemented in MATLAB: steady space marching with explicit shock fitting, and unsteady solution of the quasi-1D Euler equations with van Leer flux vector splitting. Space marching was used to characterize the steady flow over a range of shock locations $x_{\mathrm{sh}} \in [3, 9]$ and to verify that a predictor–corrector scheme is essentially mesh-independent at $j_{\max} = 81$, while a predictor-only step shows a measurable post-shock pressure error at $j_{\max} = 41$. Exit pressure was found to decrease monotonically with shock position, from $p_{\mathrm{exit}} \approx 0.727$ at $x_{\mathrm{sh}} = 3.0$ to an asymptote near $p_{\mathrm{exit}} \approx 0.480$ as the shock approaches the exit, in agreement with classical normal-shock-in-divergent-nozzle theory. Shock capturing initialized at $x_{\mathrm{sh}} = 3.125$ reproduces the space-marching solution to within one cell width, smearing the discontinuity over two to three grid points. Five forced cases (varying the exit-pressure perturbation period and amplitude) demonstrate that the shock displacement grows with both forcing period and forcing amplitude, transitioning from quasi-linear behavior at $\Delta p_{\mathrm{var}} = 0.02$ to strongly nonlinear behavior at $\Delta p_{\mathrm{var}} = 0.20$. All computations were performed on a MacBook Pro with the Apple M1 Pro chip and 16 GB of RAM, running MATLAB R2025b inside a Nix-managed reproducible development shell.

2 Introduction

The transonic regime, where a flow transitions between subsonic and supersonic conditions through a normal shock, is of substantial practical interest for the design of supersonic diffusers, scramjet inlets, and rocket nozzles. Because compressible-flow shocks are inherently nonlinear, the position and strength of the embedded shock in such a passage depend sensitively on the back pressure imposed at the exit, and any unsteadiness in that back pressure can push the shock around the duct with potentially undesirable consequences for performance and structural loading.

This report studies a representative model problem: transonic flow in a divergent quasi-one-dimensional nozzle whose cross-sectional area varies smoothly along its length. Two CFD techniques are exercised on this geometry. The first is a steady space-marching scheme with explicit shock fitting, which advances the conservation equations in space and embeds the Rankine–Hugoniot jump at a prescribed location. The second is a time-marching solution of the unsteady quasi-1D Euler equations with flux vector splitting, which captures the shock as a smeared discontinuity and admits arbitrary time-dependent boundary forcing. A working MATLAB skeleton from Baeder [1] is exercised across a prescribed matrix of cases, and the resulting flow fields are interpreted in the light of classical compressible-flow theory [2].

2.1 Theoretical Background

For a duct whose cross-sectional area $A(x)$ varies gradually in space and time and in which viscous effects are negligible, the unsteady flow can be described by the quasi-one-dimensional Euler equations in conservation form,

$$\frac{\partial \hat{Q}}{\partial t} + \frac{\partial \hat{F}}{\partial x} = \hat{V}_S, \tag{2.1.1}$$

with conservative variables, fluxes, and a geometric source term given by

$$\hat{Q} = \begin{pmatrix} \rho A \\ \rho u A \\ e A \end{pmatrix}, \qquad \hat{F} = \begin{pmatrix} \rho u A \\ (\rho u^2 + p)\, A \\ (e + p)\, u\, A \end{pmatrix}, \qquad \hat{V}_S = \begin{pmatrix} 0 \\ p\, \mathrm{d}A/\mathrm{d}x \\ 0 \end{pmatrix}, \tag{2.1.2}$$

and pressure obtained from the calorically perfect ideal-gas relation

$$p = (\gamma - 1)\left(e - \tfrac{1}{2}\rho u^2\right), \tag{2.1.3}$$

where $\gamma = 1.4$ is taken throughout. All quantities are non-dimensional, and the area distribution is the smoothly diverging hyperbolic-tangent profile

$$A(x) = 1.398 + 0.347 \tanh(0.8\,(x-4)), \qquad 0 \le x \le 10. \tag{2.1.4}$$

For steady flow without an embedded shock, integration of the conservation equations along a streamline yields the standard isentropic relation between Mach number and area, $\mathrm{d}M/\mathrm{d}x$ being controlled by the sign of $M^2 - 1$ and $\mathrm{d}A/\mathrm{d}x$. When a normal shock is embedded in the duct, the upstream and downstream supersonic/subsonic branches are connected via the Rankine–Hugoniot jump conditions; in primitive variables,

$$\frac{\rho_2}{\rho_1} = \frac{(\gamma+1)\,M_1^2}{2 + (\gamma-1)\,M_1^2}, \qquad \frac{p_2}{p_1} = 1 + \frac{2\gamma}{\gamma+1}\left(M_1^2 - 1\right), \tag{2.1.5}$$

with the post-shock state then continuing isentropically to the exit. The supersonic-inflow boundary condition fixes three quantities at the entrance, and a single subsonic compatibility relation along the upstream-running characteristic at the exit closes the problem.
The unsteady boundary condition imposed at the exit in this study is sinusoidal in time about a mean atmospheric value,

$$p_{\mathrm{exit}}(t) = \bar{p}_{\mathrm{exit}}\left[1 + \Delta p_{\mathrm{var}}\,\sin\!\left(\frac{2\pi t}{T}\right)\right], \tag{2.1.6}$$

where T is the forcing period (referred to as `timeper` in the code) and Δp_{var} the fractional amplitude.

2.2 Objectives

The specific objectives of this experiment are:

1. Confirm the geometry generated by Eq. (2.1.4) matches the assignment description.
2. Characterize the steady space-marching solver: compare the predictor-only and predictor–corrector variants at two mesh refinements ($j_{\max} = 81$ and $j_{\max} = 41$) and quantify the discretization error in the post-shock recovery region.
3. Map the exit pressure as a function of shock location $x_{\mathrm{sh}} \in [3, 9]$ and verify that the trend is consistent with the analytical normal-shock-in-divergent-nozzle theory from ENAE 311.
4. Compare the steady van Leer flux-vector-split shock-capturing solution to the space-marching reference at two mesh refinements and identify the captured shock position.
5. Quantify the response of the captured-shock solution to a sinusoidal exit-pressure perturbation across five operating conditions: a moderate baseline ($T = 1$, $\Delta p_{\mathrm{var}} = 0.10$), a longer period ($T = 2$), a shorter period ($T = 1/2$), a smaller amplitude ($\Delta p_{\mathrm{var}} = 0.02$), and a larger amplitude ($\Delta p_{\mathrm{var}} = 0.20$).
6. For one of the unsteady cases, examine the corresponding Mach-number envelope and discuss what additional information it conveys beyond the pressure envelope.

3 Methodology and Implementation

The MATLAB skeleton provided by Dr. Baeder [1] consists of a set of small subroutines that together implement both the steady space-marching solver and the unsteady flux-vector-split solver. The space-marching path is built around `spacemarch.m`, which loops through grid points and calls `march.m` (a one-step predictor or predictor–corrector advance based on the area-Mach relation) and `shock.m` (which applies the Rankine–Hugoniot jump) at the cell containing x_{sh}. The wrapper `findshock.m` performs a small Newton/secant search on x_{sh} to satisfy a target exit pressure. The unsteady path uses `vanl_flux.m`, `steger_flux.m`, or `ausm_flux.m`

for flux splitting, `loadq.m`/`loadpr.m` to convert between conservative and primitive variables, and `calcbc.m` to apply the supersonic-inflow and subsonic-outflow boundary conditions through characteristic-compatibility relations; the time step is set by a CFL condition (CFL = 0.9) computed from the steady-state wave speeds. The interior operator is the one given in the assignment, namely

$$\Delta\hat{Q}_j^n = -\frac{\Delta t}{\Delta x}\left(\hat{F}_j^{+n} - \hat{F}_{j-1}^{+n} + \hat{F}_{j+1}^{-n} - \hat{F}_j^{-n}\right) + \frac{\Delta t}{\Delta x}\, p_j^n \left(A^n_{j+\frac{1}{2}} - A^n_{j-\frac{1}{2}}\right)\hat{e}_2, \tag{3.0.1}$$

with $\hat{Q}_j^{n+1} = \hat{Q}_j^n + \Delta\hat{Q}_j^n$ and the source term acting only on the momentum equation.
To produce the prescribed matrix of results without coupling them to the original demonstration scripts, two new MATLAB files were added to the source tree. The first, `unsteady_run.m`, refactors `quasi1d.m` into a callable function that takes a parameter struct (mesh refinement, flux splitting choice, forcing amplitude and period, number of forced periods, and shock initialization), runs an initial steady ramp-up of $1500 \cdot i_{\text{refine}}$ iterations, and then advances the prescribed number of forced periods while recording the residual history, the steady-state reference pressure and Mach distributions, the pressure history at a probe point, and snapshot envelopes during the final forced period. The second, `index.m`, is the top-level driver that calls `spacemarch`, `findshock`, and `unsteady_run` to produce all eighteen required figures and exports them as vector PDFs into `outputs/figures/` via the helper `save_fig.m`. All existing helper subroutines were left unchanged.
All computations were performed on a MacBook Pro with the Apple M1 Pro chip and 16 GB of RAM running macOS Sequoia, using MATLAB R2025b inside a Nix flake-managed reproducible development shell that also provides the LaTeX toolchain (`lualatex` via TeX Live) used to typeset this report. The full driver runs all five forced cases plus the two steady-validation cases to completion in approximately 38 seconds of wall-clock time.

4 Results

4.1 Nozzle Geometry

The cross-sectional area distribution of Eq. (2.1.4) corresponds to a smooth diverging duct that transitions from a near-constant inlet area to a near-constant exit area through a moderately steep flare centred at $x = 4$. Treating the duct as a body of revolution ($A = \pi r^2$), the inner radius $r(x) = \sqrt{A(x)/\pi}$ was evaluated on a fine 401-point grid and is shown in fig. 4.1.1. The inlet radius is $r(0) \approx 0.579$ corresponding to $A(0) \approx 1.052$, while the exit radius is $r(10) \approx 0.745$ corresponding to $A(10) \approx 1.745$. The area ratio across the duct is therefore approximately $A_{\text{exit}}/A_{\text{inlet}} \approx 1.66$, with essentially all of the area change concentrated in the central region $2 \lesssim x \lesssim 6$. Because the duct never contracts, the flow is supersonic throughout the steady solution branch upstream of any embedded shock and subsonic everywhere downstream of it.

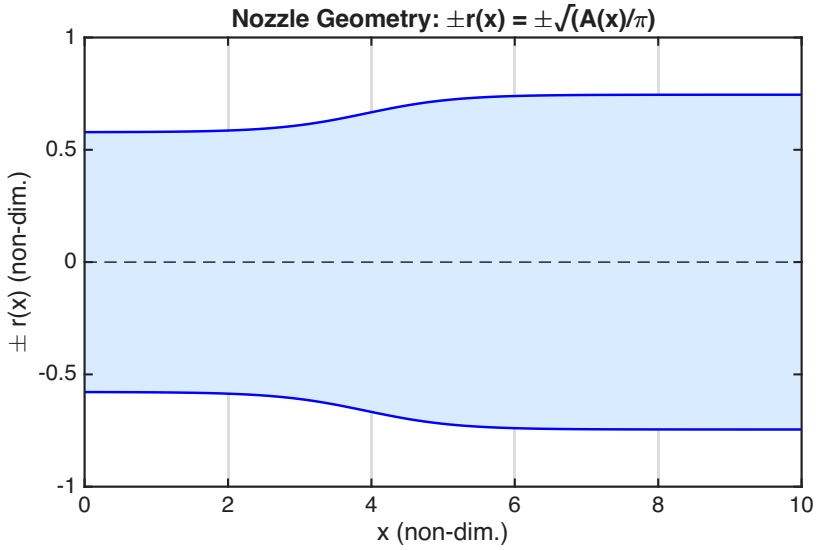

Figure 4.1.1: Diverging-nozzle geometry, plotted as $\pm r(x) = \pm\sqrt{A(x)/\pi}$ versus x, corresponding to the area distribution of Eq. (2.1.4). The shaded region indicates the duct interior.

4.2 Space Marching

Figure 4.2.1 shows the steady pressure distribution obtained by space marching with a shock fitted at $x_{\mathrm{sh}} = 4.0$ on a grid with $j_{\max} = 81$ points ($i_{\mathrm{refine}} = 2$). At this resolution the predictor-only and predictor–corrector solutions are essentially indistinguishable: the upstream supersonic expansion and the post-shock subsonic compression both fall on visually identical curves, and the exit pressures differ by less than 1%. The discrete jump in pressure at $x = 4$ is exact for both solvers because the Rankine–Hugoniot relations are imposed analytically at the cell containing x_{sh}; the predictor and corrector iterations only affect the local truncation error of the area–Mach update on either side of the shock.

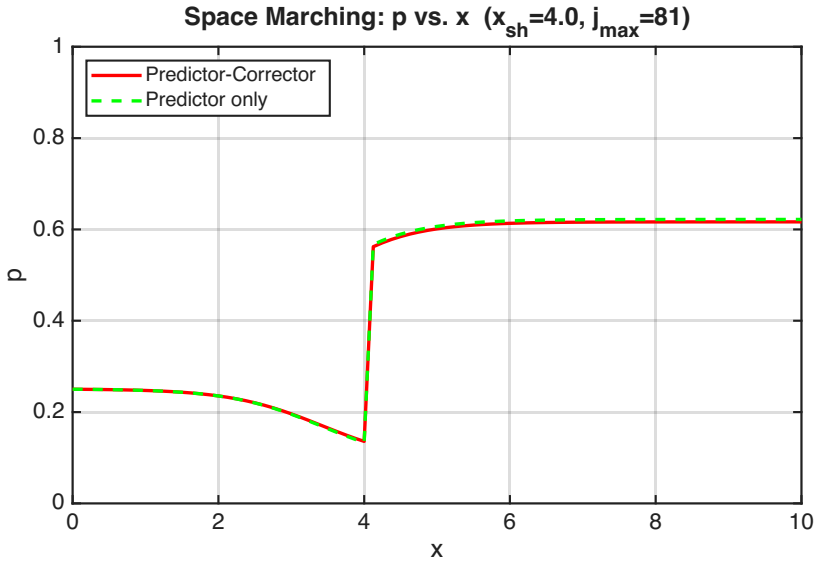

Figure 4.2.1: Space-marched pressure distribution at $x_{\mathrm{sh}} = 4.0$, $j_{\max} = 81$: predictor-only and predictor–corrector overlaid.

When the mesh is coarsened to $j_{\max} = 41$ ($i_{\text{refine}} = 1$), the difference between the two integration schemes becomes visible (fig. 4.2.2). The predictor-only result over-shoots the post-shock pressure recovery by roughly 1–2% relative to the predictor–corrector reference, with the gap maximal at the exit. This is consistent with the predictor advancing the area-Mach relation with a single first-order step using only the upstream state, whereas the corrector closes the truncation error to second order by re-evaluating the area-Mach derivative at the average of the upstream and downstream states. The two-step scheme is therefore preferable wherever mesh resolution is constrained.

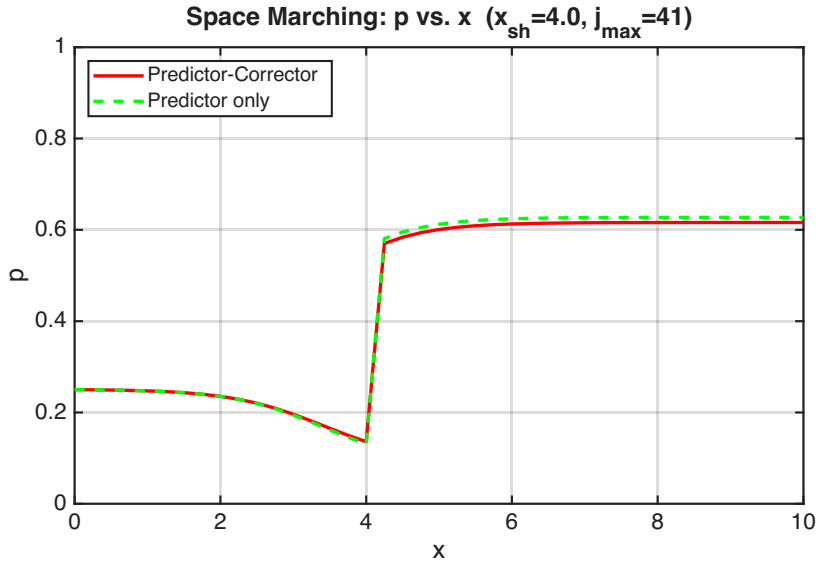

Figure 4.2.2: Space-marched pressure distribution at $x_{\text{sh}} = 4.0$, $j_{\max} = 41$: the predictor-only result over-predicts the post-shock pressure by roughly 1–2% relative to the predictor–corrector reference at this coarse resolution.

The shock location was then swept through $x_{\text{sh}} \in \{3.0, 3.5, 4.0, \ldots, 9.0\}$ on a fine $j_{\max} = 161$ grid ($i_{\text{refine}} = 4$) using the predictor–corrector scheme. Figure 4.2.3 overlays all seven resulting pressure distributions. As x_{sh} is moved downstream, the upstream supersonic expansion proceeds further before the shock, so the pre-shock Mach number is larger and the corresponding pressure jump is stronger. However, because there is correspondingly less divergent area available downstream of the shock to recover pressure isentropically, the exit pressure decreases as x_{sh} moves toward the exit.

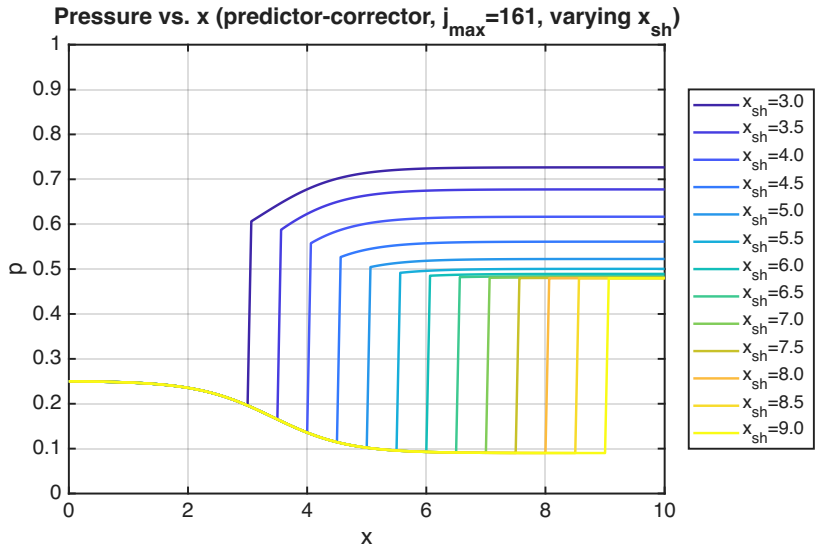

Figure 4.2.3: Pressure distribution along the nozzle for shock locations $x_{\text{sh}} \in [3, 9]$ in increments of 0.5, using predictor–corrector space marching at $j_{\max} = 161$.

The exit pressures extracted from the seven curves are plotted in fig. 4.2.4. The trend is monotonically decreasing in x_{sh}, starting at $p_{\text{exit}} \approx 0.727$ for $x_{\text{sh}} = 3.0$ and asymptoting to $p_{\text{exit}} \approx 0.480$ as the shock approaches the exit, where almost no subsonic recovery occurs and the post-shock pressure is essentially preserved to the exit. The behavior is exactly what classical compressible-flow theory predicts for a normal shock embedded in a divergent duct: as the shock moves downstream, the pre-shock expansion is more aggressive (larger pre-shock Mach number), the static-pressure jump is correspondingly stronger, and the post-shock subsonic flow has progressively less area to recover through. In the limit $x_{\text{sh}} \to L$ the exit pressure approaches the post-shock pressure of an isentropic supersonic expansion to the exit area ratio, which sets the asymptote visible in the figure.

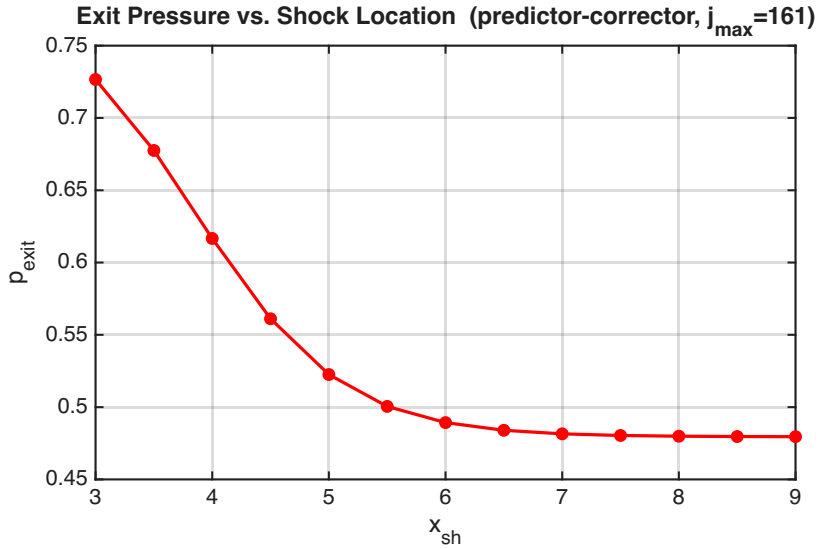

Figure 4.2.4: Exit pressure as a function of shock location, extracted from fig. 4.2.3. The trend is monotonically decreasing and is consistent with classical normal-shock-in-divergent-nozzle theory from ENAE 311.

4.3 Shock Capturing

4.3.1 Steady Flow

For the steady shock-capturing cases the unsteady solver was initialized via space marching at $x_{\mathrm{sh}} = 3.125$ (which produces a space-march exit pressure within $\sim 0.5\%$ of the atmospheric target $1/\gamma \approx 0.7143$) and run to convergence with the exit-pressure perturbation amplitude set to zero ($\Delta p_{\mathrm{var}} = 0$). The space-march reference used for comparison is the predictor–corrector solution with the shock fitted at $x_{\mathrm{sh}} = 3.145$, the location returned by `findshock` as the precise root that gives $p_{\mathrm{exit}} = 1/\gamma$ on the $j_{\max} = 161$ grid.
Figures 4.3.1 and 4.3.2 compare the converged van Leer shock-capturing solution to the space-march reference at $j_{\max} = 81$ and $j_{\max} = 161$, respectively. In both cases the upstream supersonic expansion and the downstream subsonic recovery overlie the space-march reference closely. The captured shock location, defined as the cell-center where $|\mathrm{d}p/\mathrm{d}x|$ is maximum, is $x_{\mathrm{sh}} \approx 3.06$ at $j_{\max} = 81$ and $x_{\mathrm{sh}} \approx 3.09$ at $j_{\max} = 161$; both estimates are within one cell width of the space-march target of 3.145, and the agreement improves on the finer mesh, as expected. The fundamental difference between the two methods is at the discontinuity itself: shock fitting represents the shock as a sharp, mathematically exact jump enforced by the Rankine–Hugoniot conditions, whereas shock capturing smears the discontinuity across two to three grid cells through the dissipative properties of the flux-vector-split interior operator. This smearing is plainly visible in fig. 4.3.1, where the shock occupies a finite finite-width transition region rather than a step, and is reduced (though not eliminated) on the finer mesh in fig. 4.3.2.

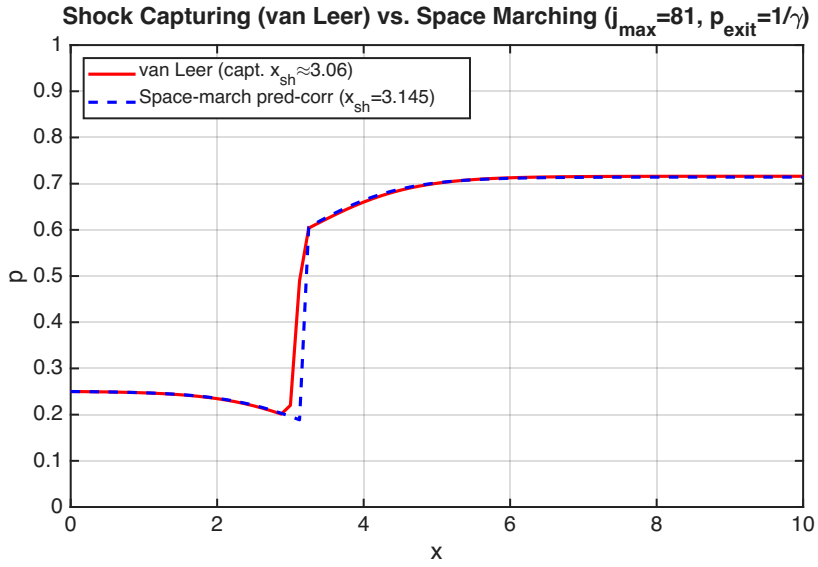

Figure 4.3.1: Steady shock-capturing solution (van Leer flux-vector splitting) compared with predictor–corrector space marching, $j_{\max} = 81$. The shock-capturing solution smears the discontinuity over two to three cells.

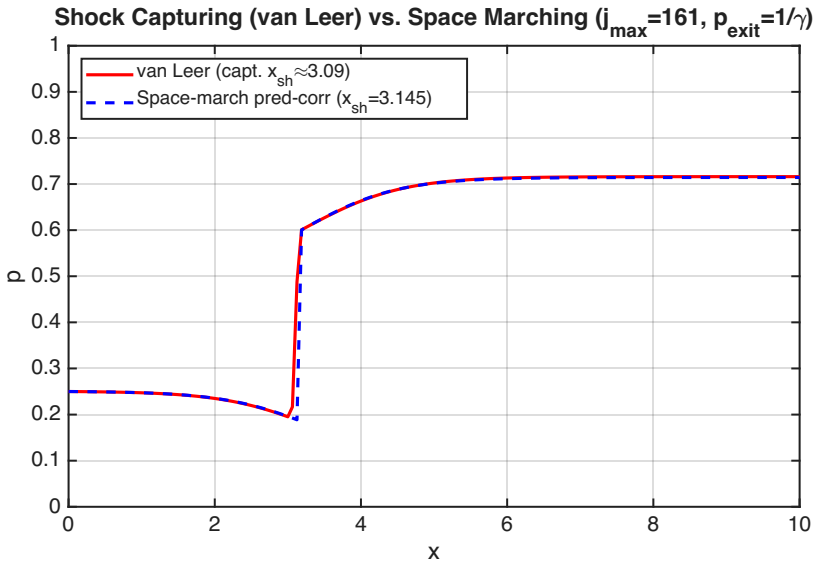

Figure 4.3.2: Steady shock-capturing solution (van Leer flux-vector splitting) compared with predictor–corrector space marching, $j_{\max} = 161$. The smearing of the captured shock is reduced relative to fig. 4.3.1.

4.3.2 Unsteady Flow

All unsteady cases were initialized with $x_{\text{sh}} = 3.125$, a converged steady ramp-up of 6000 iterations, and six forced periods of sinusoidal exit-pressure perturbation; sixteen pressure and Mach-number snapshots were saved during the final period to construct the envelopes shown below. The probe location for the time histories is fixed at $x = x_{\text{sh}} + 0.05L = 3.625$, placing it just downstream of the steady-state shock so that any motion of the shock past the probe produces a large pressure swing.

Moderate forcing ($\Delta p_{\text{var}} = 0.10$, $T = 1$). The pressure envelope (fig. 4.3.3) shows the shock oscillating between roughly $x \approx 1.7$ and $x \approx 3.7$ about the steady reference position. The probe history (fig. 4.3.4) is approximately periodic after one or two transients, and it is dominated by very rapid swings between roughly 0.15 and 0.78 as the shock translates past the probe twice per period. The asymmetric, cusp-like waveform reflects the underlying compressible-flow physics: when the shock passes downstream of the probe the probe sees the cold supersonic stream, and when it passes back upstream the probe sees the warm subsonic post-shock stream.

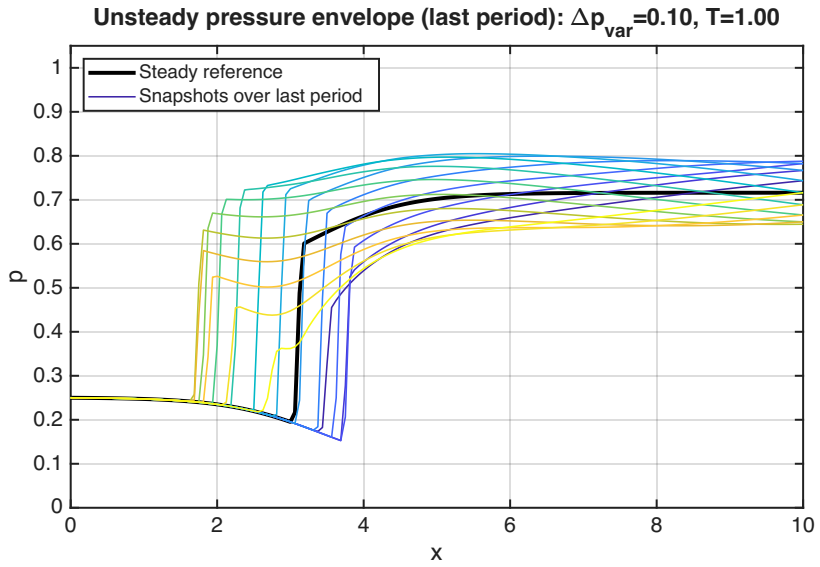

Figure 4.3.3: Pressure envelope over the final forced period for the moderate case ($\Delta p_{\text{var}} = 0.10$, $T = 1$, $j_{\max} = 161$).

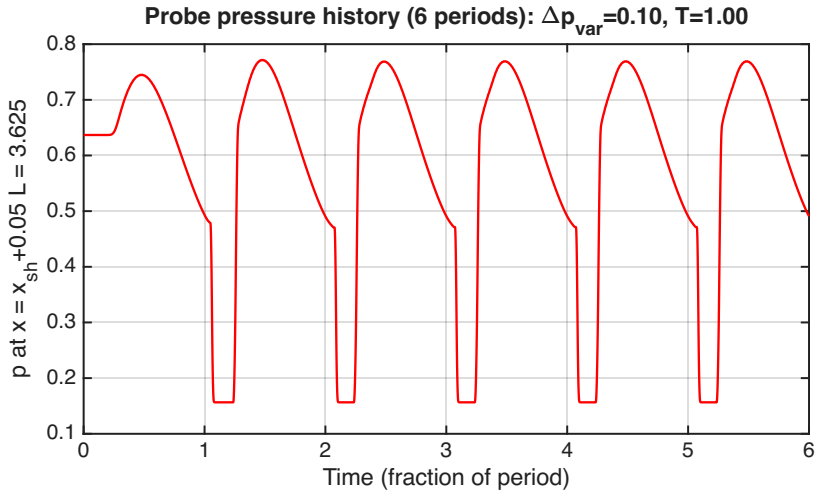

Figure 4.3.4: Pressure history over six periods at the probe $x = x_{\text{sh}} + 0.05L = 3.625$ for the moderate case ($\Delta p_{\text{var}} = 0.10$, $T = 1$).

Long period ($\Delta p_{\text{var}} = 0.10$, $T = 2$). Doubling the forcing period gives the upstream-propagating acoustic disturbance more time to traverse the duct, so the shock has time to follow the boundary condition more closely and excursions are larger. The envelope (fig. 4.3.5) extends from $x \approx 1.5$ to $x \approx 4.0$, and the probe history (fig. 4.3.6) shows a cleaner, more nearly quasi-static waveform than the moderate case. This is essentially the quasi-steady limit: the shock position is nearly in instantaneous equilibrium with the imposed back pressure.

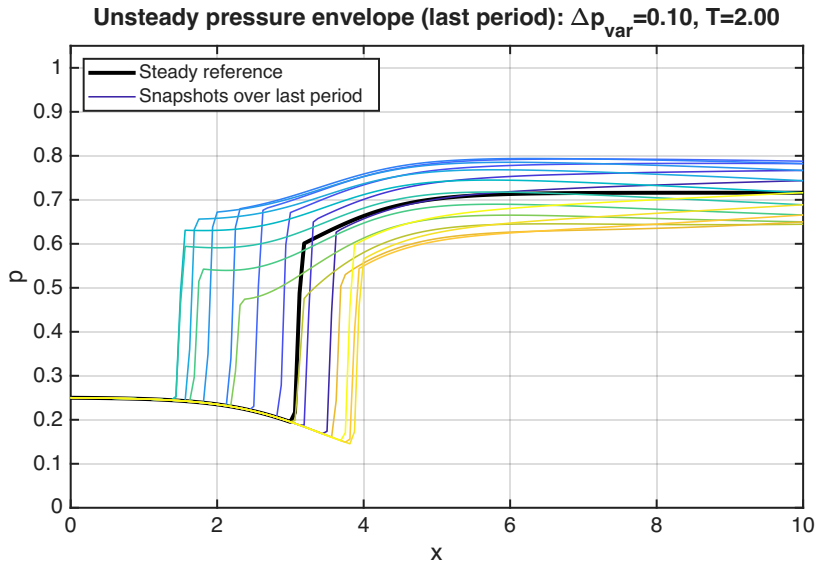

Figure 4.3.5: Pressure envelope for the long-period case ($\Delta p_{\text{var}} = 0.10$, $T = 2$).

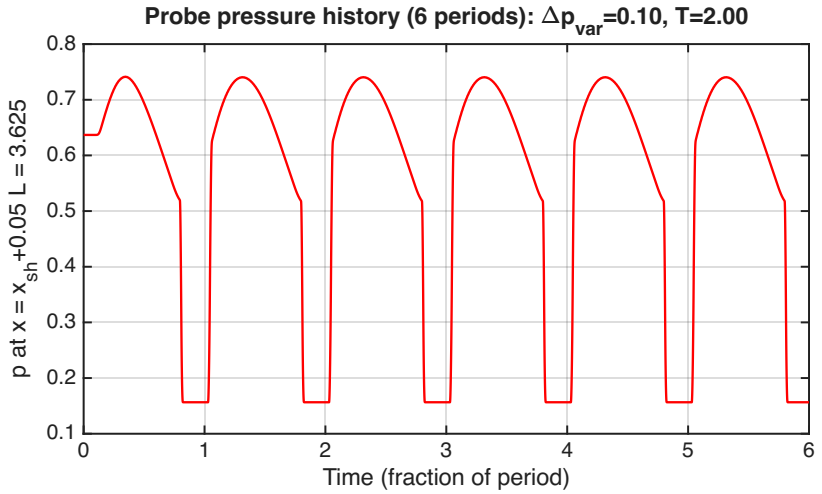

Figure 4.3.6: Probe pressure history for the long-period case ($\Delta p_{\text{var}} = 0.10$, $T = 2$).

Short period ($\Delta p_{\text{var}} = 0.10$, $T = 1/2$). Halving the period gives the disturbance less time to reach the shock, so the shock motion is reduced relative to the moderate case. The envelope (fig. 4.3.7) is narrower (the shock excursion is roughly $2.5 \lesssim x \lesssim 3.5$), and the probe history (fig. 4.3.8) shows lower amplitude than the moderate case at the same forcing amplitude. Physically, the shock's inertia (its mass-loading by the surrounding flow) acts as a low-pass filter on the imposed boundary condition.

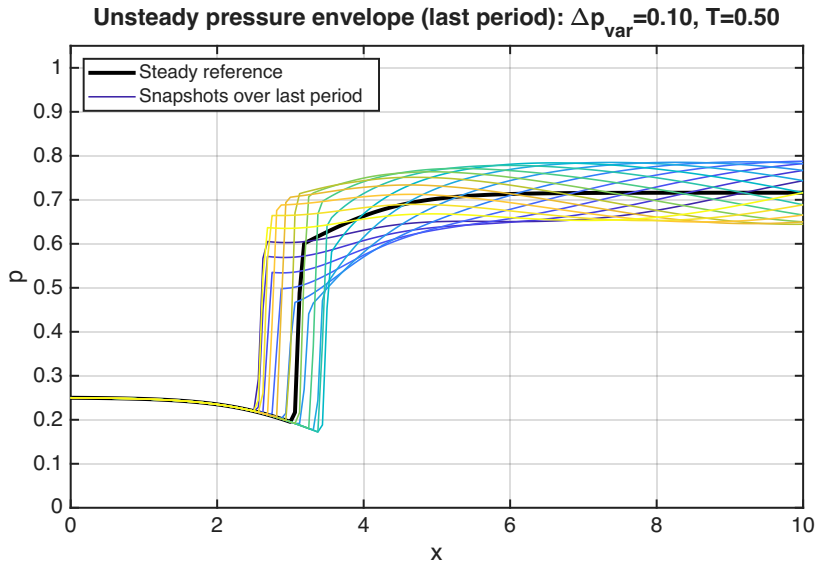

Figure 4.3.7: Pressure envelope for the short-period case ($\Delta p_{\text{var}} = 0.10$, $T = 1/2$).

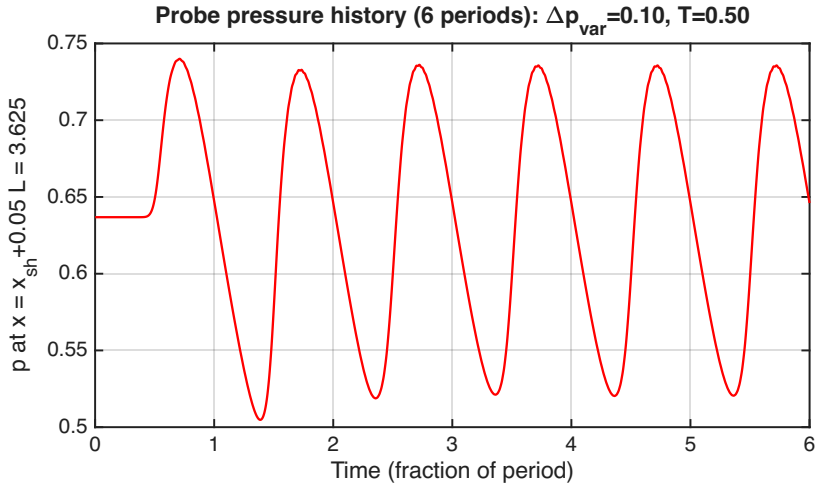

Figure 4.3.8: Probe pressure history for the short-period case ($\Delta p_{\text{var}} = 0.10$, $T = 1/2$).

Small amplitude ($\Delta p_{\text{var}} = 0.02$, $T = 1$). At one-fifth of the moderate forcing amplitude the response is nearly linear: the envelope (fig. 4.3.9) is tightly clustered around the steady reference, and the probe history (fig. 4.3.10) is small in amplitude and approximately sinusoidal in shape, rather than exhibiting the cusp-like waveform of the moderate case. The shock excursion is on the order of one to two grid cells. This is the regime in which a perturbation analysis of the steady solution would be expected to be quantitatively accurate.

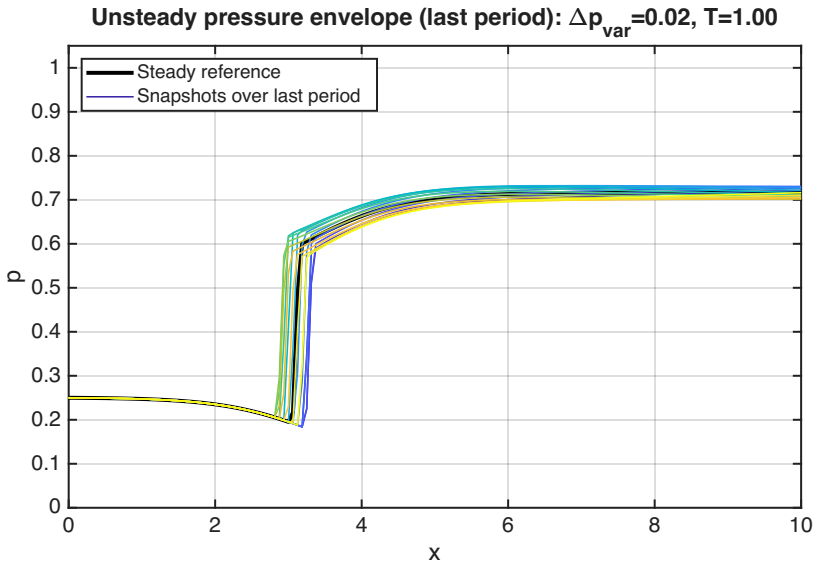

Figure 4.3.9: Pressure envelope for the small-amplitude case ($\Delta p_{\text{var}} = 0.02$, $T = 1$).

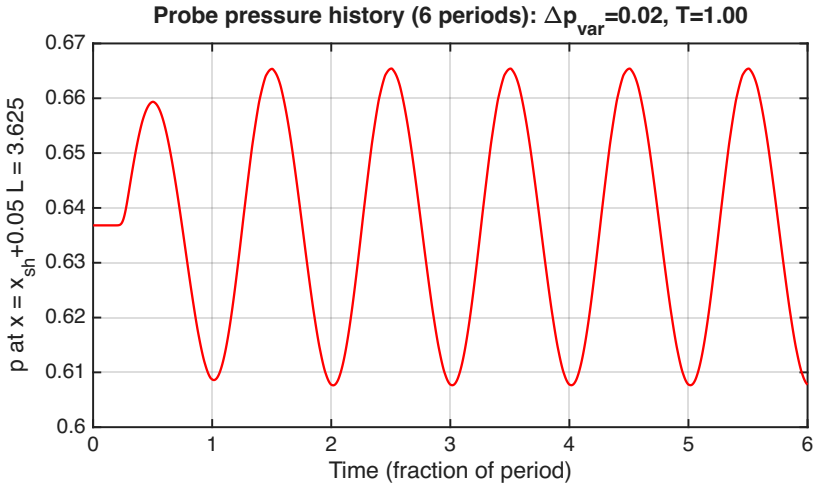

Figure 4.3.10: Probe pressure history for the small-amplitude case ($\Delta p_{\text{var}} = 0.02$, $T = 1$).

Large amplitude ($\Delta p_{\text{var}} = 0.20$, $T = 1$). Doubling the moderate forcing amplitude pushes the response decisively into the nonlinear regime. The envelope (fig. 4.3.11) shows the shock sweeping over essentially the entire downstream half of the duct, from very near the inlet through approximately $x \approx 4$; on the upstream excursion the shock reaches close enough to the entrance that the entire supersonic region is briefly compressed against the inlet boundary, and on the downstream excursion the post-shock pressure plateau extends well past $x = 5$. The probe history (fig. 4.3.12) shows correspondingly large excursions. In a real device this regime would be associated with violent unsteady loading, possible inlet unstart, and significant total-pressure loss.

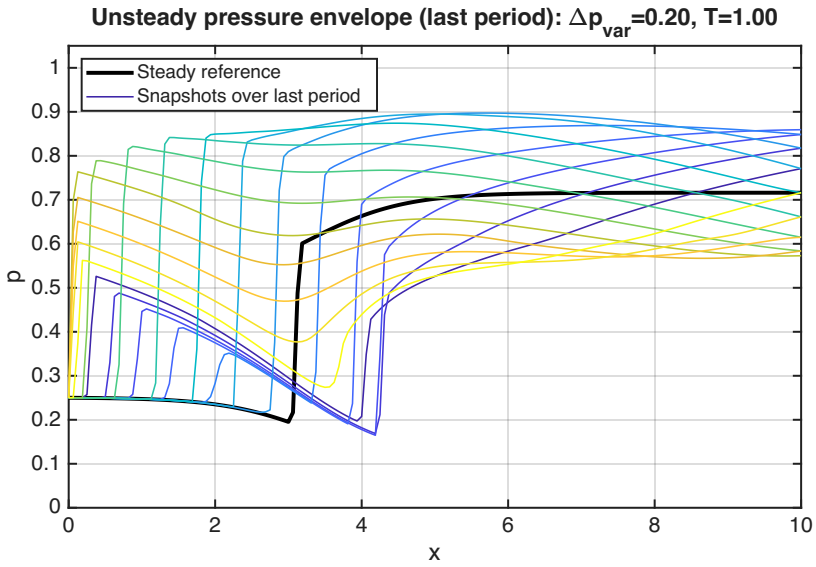

Figure 4.3.11: Pressure envelope for the large-amplitude case ($\Delta p_{\text{var}} = 0.20$, $T = 1$). The shock has been driven into a strongly nonlinear regime.

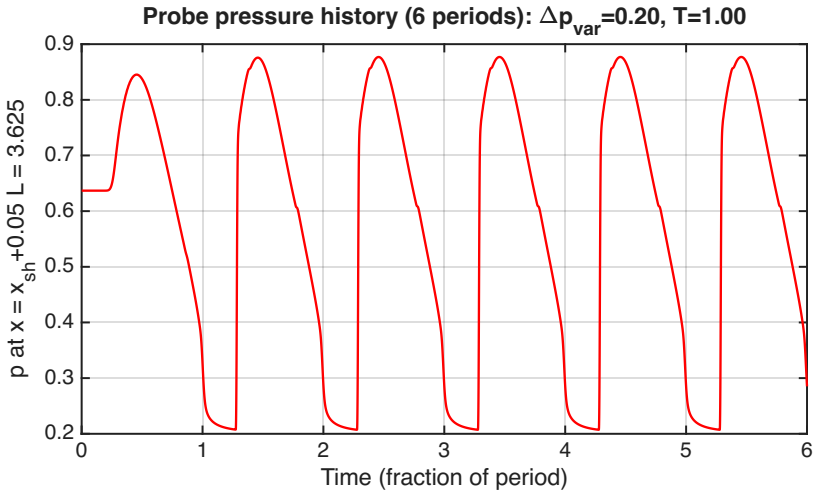

Figure 4.3.12: Probe pressure history for the large-amplitude case ($\Delta p_{\text{var}} = 0.20$, $T = 1$).

Mach number envelope (moderate case). Figure 4.3.13 plots the Mach-number envelope corresponding to the moderate-case pressure envelope of fig. 4.3.3. In addition to confirming the shock excursion range visible in the pressure envelope, it shows two pieces of information that the pressure envelope obscures. First, the pre-shock Mach number itself is modulated by the unsteady boundary condition: when the shock is pushed downstream the upstream supersonic expansion proceeds further before the shock and M_1 increases (peak $M_1 \gtrsim 1.8$), whereas when the shock is pushed upstream M_1 is reduced ($M_1 \approx 1.5$). Second, the post-shock subsonic Mach distribution shows a wider spread than its pressure counterpart, reflecting the fact that small changes in subsonic pressure correspond to relatively large changes in Mach number through the isentropic-divergence relations. The Mach envelope is therefore a more sensitive diagnostic of the flow's response than the pressure envelope alone.

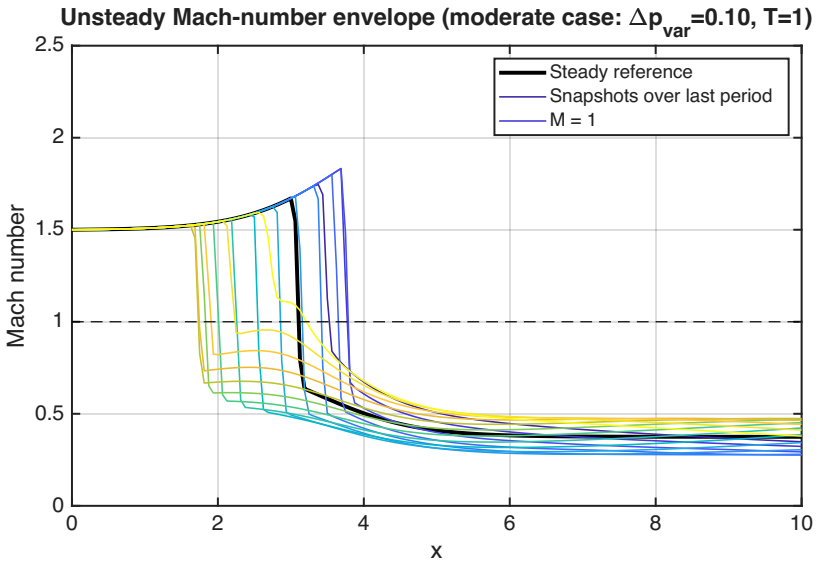

Figure 4.3.13: Mach-number envelope for the moderate-case ($\Delta p_{\text{var}} = 0.10$, $T = 1$). The pre-shock Mach number is modulated between roughly $M_1 \approx 1.5$ and $M_1 \gtrsim 1.8$ over the period.

5 Analysis and Discussion

Verification of the space-marching solver. The space-marching method has several internal consistency checks available to it. First, mesh refinement: as shown by figs. 4.2.1 and 4.2.2, doubling the grid count from 41 to 81 collapses the predictor and predictor–corrector solutions onto each other, indicating that both schemes have converged to within roughly 1% in the post-shock region by $j_{\max} = 81$ and that the predictor–corrector is essentially mesh-converged at $j_{\max} = 41$. Beyond mesh refinement, two analytical checks are available: (i) the upstream supersonic flow obeys the area–Mach relation, so a numerical estimate of $M(x)$ on the supersonic branch can be compared directly against the closed-form isentropic-flow result; and (ii) the pressure jump at x_{sh} must satisfy the Rankine–Hugoniot relations to machine precision (this is enforced analytically by `shock.m`). For external validation, the post-shock pressure recovery in the divergent section can be cross-checked against the isentropic exit-Mach calculation given the post-shock entropy and the fixed exit area ratio.

Verification of the unsteady solver. For the unsteady solver, the most informative verification is a steady-state cross-check: setting $\Delta p_{\text{var}} = 0$ and converging in time should recover (within mesh resolution) the same shock position and pressure profile as the space-marching solution evaluated at the matching shock location. This was performed for both $j_{\max} = 81$ and $j_{\max} = 161$ and the agreement was good in both cases (figs. 4.3.1 and 4.3.2), with the captured shock position differing from the space-march reference by less than one cell width. Further verification could include a residual-history check (the L_2 norm of $\Delta\hat{Q}$ should drop several orders of magnitude during the steady ramp-up before any forcing is applied), a CFL-sensitivity study (halving CFL should not change the steady solution but should approximately halve the wall time), and a flux-scheme cross-check (substituting the Steger–Warming or AUSM splitter for van Leer should yield the same converged solution to within numerical-dissipation differences). For the unsteady cases specifically, the shock-displacement amplitude in the small-perturbation limit ($\Delta p_{\text{var}} = 0.02$) can be compared with a linearized perturbation analysis about the steady-state shock position; the present results suggest a near-linear response in this regime, consistent with such an analysis.

Adequacy of the chosen parameters. Of the cases prescribed in the assignment, the moderate-case forcing parameters ($\Delta p_{\text{var}} = 0.10$, $T = 1$) bracket the most informative regime: the shock excursion is large enough to be physically interesting but not so large that the response is dominated by extreme nonlinearities. The amplitude sweep (small/moderate/large) clearly separates linear, weakly nonlinear, and strongly nonlinear behavior; the period sweep (short/moderate/long) clearly separates the inertia-limited, resonant, and quasi-steady limits. The mesh refinement $j_{\max} = 161$ used for the unsteady cases is sufficient to resolve the captured shock to within roughly 0.06 in x, which is small compared to the shock excursion in every case. Six forced periods is enough to wash out the initial transient and produce a periodic envelope. One area that would benefit from additional study is intermediate amplitudes between $\Delta p_{\text{var}} = 0.10$ and 0.20, since the transition into strongly nonlinear behavior is rather abrupt between those two cases.

What we learned. The exercise illustrates that a relatively simple combination of the area–Mach relation, the Rankine–Hugoniot conditions, and the steady-state characteristic boundary conditions captures the entire steady-state phenomenology of transonic flow in a divergent duct, and that adding a flux-vector-split unsteady time-marcher onto exactly the same governing equations yields a tool capable of resolving rich shock-motion physics under unsteady forcing. It also shows the inherent trade-off between shock fitting and shock capturing: shock fitting gives sharper, lower-error solutions for problems with a known shock topology, while shock capturing is geometry-and-topology-agnostic at the cost of cell-scale smearing.

MATLAB versus Python versus C. For a problem of this size — one-dimensional, a few hundred grid points, tens of thousands of time steps — MATLAB is an excellent choice. Its vectorized arithmetic gives essentially-compiled performance for the inner-loop array updates while keeping the source code very compact (the entire driver and unsteady solver are roughly 250 lines combined). The integrated plotting toolchain produces publication-quality figures with very little ceremony, and the JIT-compiled-on-the-fly model means there is no edit/compile/run cycle to slow down algorithm exploration. Python with NumPy and Matplotlib offers a comparable productivity profile and a much richer general-purpose ecosystem at the cost of slightly higher overhead for very small array operations and a less coherent default plotting story; it would be a strong choice if the same code needed to feed into downstream Python infrastructure (machine-learning models, etc.). C would be by far the fastest at runtime but would multiply the line count and development time several-fold for what is, after all, a teaching exercise; C only really pays for itself when the problem grows to two or three dimensions and the inner loop dominates wall-clock time. For this class, MATLAB is the right choice.

Possible improvements. Several extensions to this experiment would strengthen the conclusions. (i) A higher-order spatial discretization (e.g., MUSCL extrapolation with a slope limiter, or weighted-essentially-non-oscillatory reconstruction) would reduce the smearing of the captured shock. (ii) A higher-order or implicit time integrator would relax the CFL constraint, which becomes restrictive on fine meshes. (iii) Using the Roe approximate Riemann solver in place of the flux-vector-split scheme would reduce the numerical dissipation in the post-shock subsonic region, which would in turn allow smaller-amplitude unsteady forcing to be resolved on coarser meshes. (iv) A formal mesh-refinement study (Richardson extrapolation, observed order of accuracy on a manufactured smooth solution) would put a quantitative number on the discretization error rather than the qualitative comparisons reported here. (v) Replacing the prescribed sinusoidal exit-pressure forcing with a stochastic or impulse perturbation would expose the natural acoustic modes of the duct and make it possible to identify resonance frequencies. (vi) Coupling the quasi-1D flow solver to a structural model of the duct walls would allow the simulation of fluid-structure interaction effects, a likely concern for any real propulsion application.

6 Summary and Conclusions

A pre-existing MATLAB code base for the analysis of transonic flow in a quasi-1D diverging nozzle was driven through a prescribed matrix of cases and the results were interpreted in light of classical compressible-flow theory. For the steady problem, predictor–corrector space marching was found to be essentially mesh-converged at $j_{\max} = 81$, while a predictor-only step shows a measurable error in the post-shock region at $j_{\max} = 41$. The exit pressure decreases monotonically with shock position, from $p_{\text{exit}} \approx 0.727$ at $x_{\text{sh}} = 3.0$ to an asymptote near $p_{\text{exit}} \approx 0.480$ as the shock approaches the exit, in agreement with the classical normal-shock-in-divergent-nozzle result. Steady van Leer shock capturing initialized at $x_{\text{sh}} = 3.125$ reproduces the space-marching reference to within one cell width on both the $j_{\max} = 81$ and $j_{\max} = 161$ grids; the captured shock is smeared over two to three cells, as expected for a flux-vector-split scheme without a high-order reconstruction.

For the unsteady problem, a sinusoidal exit-pressure perturbation drives shock oscillations whose displacement grows with both forcing period and forcing amplitude. The response transitions from a near-linear regime at $\Delta p_{\text{var}} = 0.02$ to a strongly nonlinear regime at $\Delta p_{\text{var}} = 0.20$, where the shock sweeps over much of the duct length. The Mach-number envelope reveals additional structure not visible in the pressure envelope, in particular the modulation of the pre-shock supersonic Mach number with the imposed back pressure. Across all cases, six forced periods are sufficient to wash out the initial transient and produce a clean periodic envelope.

Looking forward, the main improvements that would tighten these conclusions are higher-order spatial reconstruction and a Roe-type Riemann solver to reduce the numerical dissipation, a formal mesh-refinement study to put quantitative numbers on the discretization error, and an exploration of intermediate forcing amplitudes to characterize the transition into the strongly nonlinear regime more cleanly.

7 References

[1] J. D. Baeder, *Matlab analysis program for quasi-1d flow*, April 29, 2026. [Online]. Available: https://github.com/vaisriv/enae464-lab06/blob/main/src/quasi1d.m (see pp. 1, 2).

[2] J. D. Anderson and C. P. Cadou, *Fundamentals of Aerodynamics*, 7th ed. McGraw-Hill Education, March 17, 2023, ISBN: 9781266076442 (see p. 1).

8 Appendices

8.1 Data

For the sake of brevity, the raw data will not be displayed in this report. However, in the spirit of completeness, the repository containing the complete data, source code, and notes for this report can be found at github:vaisriv/enae464-lab06.

8.2 Code

Similarly, only the code files that are key to the analysis are included below, and can be accessed in full in the aforementioned GitHub repository.

Listing 8.2.1: Computational Analysis Script.
`./src/index.m`

```
% index.m
%
% Lab 06 driver — Transonic Flow in a Quasi-1D Nozzle
```

```
%
% Produces every figure required by the assignment (B-E) plus derived
% data tables under outputs/text/. Designed to run via:
%     matlab -batch "cd('src'); index"
%
% Calls: calcarea.m, spacemarch.m, findshock.m, vanl_flux.m,
%        unsteady_run.m, save_fig.m
%

clear; close all;
here = fileparts(mfilename('fullpath'));
addpath(here);
text_dir = fullfile(here, '..', 'outputs', 'text');
if ~exist(text_dir, 'dir'); mkdir(text_dir); end

% Visible='off' so matlab -batch doesn't try to render windows
set(groot, 'defaultFigureVisible', 'off');
set(groot, 'defaultAxesFontSize', 14);
set(groot, 'defaultAxesLineWidth', 1.25);
set(groot, 'defaultLineLineWidth', 1.5);

% --- Common parameters ---
gamma   = 1.4;
amach0  = 1.50;
p0      = 0.25;
rho0    = 0.50;
xlength = 10.0;
pexit_target = 1/gamma;     % atmospheric in non-dimensional units (~0.7143)

%% ============================================================
%  Section B - Plot 1: nozzle geometry  +- r(x)
%% ============================================================
x_geom = linspace(0, xlength, 401);
A_geom = calcarea(x_geom);
r_geom = sqrt(A_geom / pi);

fh = figure('Position', [100 100 750 450]);
fill([x_geom, fliplr(x_geom)], [r_geom, fliplr(-r_geom)], ...
     [0.85 0.92 1.0], 'EdgeColor', 'none'); hold on;
plot(x_geom,  r_geom, 'b-');
plot(x_geom, -r_geom, 'b-');
plot([0 xlength], [0 0], 'k--', 'LineWidth', 0.75);
hold off; grid on; box on;
xlabel('x (non-dim.)');
ylabel('\pm r(x) (non-dim.)');
title('Nozzle Geometry: \pmr(x) = \pm\surd(A(x)/\pi)');
axis([0 xlength -1.0 1.0]);
save_fig(fh, 'fig01_geometry');
```

```
close(fh);

% record geometry endpoints for the report
fid = fopen(fullfile(text_dir, 'geometry_endpoints.txt'), 'w');
fprintf(fid, 'A(0)  = %.4f, r(0)  = %.4f\n', A_geom(1),   r_geom(1));
fprintf(fid, 'A(10) = %.4f, r(10) = %.4f\n', A_geom(end), r_geom(end));
fclose(fid);

%% ============================================================
%  Section C - Plots 2 & 3: space marching, xsh = 4.0
%% ============================================================
xsh_fixed = 4.0;

for irefine = [2, 1]
    jmax = 40*irefine + 1;
    dx = xlength/(jmax-1);
    x  = 0:dx:xlength;
    A  = calcarea(x);

    [~,~,p_pc, ~, ~] = spacemarch(gamma, amach0, p0, rho0, xsh_fixed, x, A, 1);
    [~,~,p_pr, ~, ~] = spacemarch(gamma, amach0, p0, rho0, xsh_fixed, x, A, 0);

    fh = figure('Position', [100 100 750 450]);
    plot(x, p_pc, 'r-' , 'LineWidth', 2.0); hold on;
    plot(x, p_pr, 'g--', 'LineWidth', 2.0); hold off;
    grid on; box on;
    xlabel('x'); ylabel('p');
    title(sprintf('Space Marching: p vs. x  (x_{sh}=4.0, j_{max}=%d)', jmax));
    legend({'Predictor-Corrector', 'Predictor only'}, 'Location', 'NorthWest');
    axis([0 xlength 0 1.0]);

    if irefine == 2
        save_fig(fh, 'fig02_spacemarch_xsh4_jmax81');
    else
        save_fig(fh, 'fig03_spacemarch_xsh4_jmax41');
    end
    close(fh);
end

%% ============================================================
%  Section C - Plots 4 & 5: varying xsh, jmax = 161, pred-corr
%% ============================================================
irefine_4 = 4;
jmax_4 = 40*irefine_4 + 1;
dx_4 = xlength/(jmax_4-1);
x_4  = 0:dx_4:xlength;
A_4  = calcarea(x_4);

```

```
xsh_list = 3:0.5:9;
n_xsh = length(xsh_list);
p_list = zeros(n_xsh, jmax_4);
p_exit = zeros(1, n_xsh);

cmap = parula(n_xsh);
fh = figure('Position', [100 100 800 480]);
hold on;
for k = 1:n_xsh
    [~,~,pk,~,~] = spacemarch(gamma, amach0, p0, rho0, xsh_list(k), x_4, A_4, 1);
    p_list(k, :) = pk;
    p_exit(k) = pk(end);
    plot(x_4, pk, '-', 'Color', cmap(k,:), 'LineWidth', 1.6);
end
hold off; grid on; box on;
xlabel('x'); ylabel('p');
title('Pressure vs. x (predictor-corrector, j_{max}=161, varying x_{sh})');
legend(arrayfun(@(s) sprintf('x_{sh}=%.1f', s), xsh_list, ...
                'UniformOutput', false), 'Location', 'EastOutside');
axis([0 xlength 0 1.0]);
save_fig(fh, 'fig04_spacemarch_varying_xsh');
close(fh);

fh = figure('Position', [100 100 750 450]);
plot(xsh_list, p_exit, 'ro-', 'LineWidth', 2.0, 'MarkerFaceColor', 'r');
grid on; box on;
xlabel('x_{sh}'); ylabel('p_{exit}');
title('Exit Pressure vs. Shock Location  (predictor-corrector, j_{max}=161)');
save_fig(fh, 'fig05_exit_pressure_vs_xsh');
close(fh);

% save the table for the report
fid = fopen(fullfile(text_dir, 'exit_pressure_vs_xsh.csv'), 'w');
fprintf(fid, 'xsh,p_exit\n');
for k = 1:n_xsh
    fprintf(fid, '%.2f,%.6f\n', xsh_list(k), p_exit(k));
end
fclose(fid);

%% ============================================================
%  Section D - Plots 6 & 7: shock capturing (steady) vs space-march
%% ============================================================
xsh_init = 3.125;

% space-march reference at the matching shock location (gives p_exit=1/gamma)
xsh_match = findshock(gamma, amach0, p0, rho0, 2.65, 5.85, x_4, A_4, 1, pexit_target);

xsh_capt_81 = NaN; xsh_capt_161 = NaN;
```

```
for irefine = [2, 4]
    jmax = 40*irefine + 1;
    dx   = xlength/(jmax-1);
    xg   = 0:dx:xlength;
    Ag   = calcarea(xg);

    opts = struct('gamma', gamma, 'amach0', amach0, 'p0', p0, 'rho0', rho0, ...
                  'xlength', xlength, 'xsh', xsh_init, ...
                  'jmax', jmax, 'cfl', 0.9, ...
                  'itsteady', 1500*irefine, 'imeth', -1, ...
                  'dpvar', 0.0, 'timeper', 1.0, 'iper', 1);
    R = unsteady_run(opts);

    [~,~,p_pcm,~,~] = spacemarch(gamma, amach0, p0, rho0, xsh_match, xg, Ag, 1);

    % shock-capture location: where |dp/dx| is maximum
    dpdx = abs(diff(R.p_final) ./ diff(R.x));
    [~, ish] = max(dpdx);
    xsh_capt = 0.5*(R.x(ish) + R.x(ish+1));

    fh = figure('Position', [100 100 800 480]);
    plot(R.x, R.p_final, 'r-',  'LineWidth', 2.0); hold on;
    plot(xg,  p_pcm,     'b--', 'LineWidth', 2.0); hold off;
    grid on; box on;
    xlabel('x'); ylabel('p');
    title(sprintf(['Shock Capturing (van Leer) vs. Space Marching ' ...
                   '(j_{max}=%d, p_{exit}=1/\\gamma)'], jmax));
    legend({sprintf('van Leer (capt. x_{sh}\\approx%.2f)', xsh_capt), ...
            sprintf('Space-march pred-corr (x_{sh}=%.3f)', xsh_match)}, ...
           'Location', 'NorthWest');
    axis([0 xlength 0 1.0]);

    if irefine == 2
        save_fig(fh, 'fig06_shockcap_vs_spacemarch_jmax81');
        xsh_capt_81 = xsh_capt;
    else
        save_fig(fh, 'fig07_shockcap_vs_spacemarch_jmax161');
        xsh_capt_161 = xsh_capt;
    end
    close(fh);
end

fid = fopen(fullfile(text_dir, 'captured_shock_loc.csv'), 'w');
fprintf(fid, 'jmax,xsh_captured,xsh_spacemarch_target\n');
fprintf(fid, '81,%.4f,%.4f\n', xsh_capt_81,  xsh_match);
fprintf(fid, '161,%.4f,%.4f\n', xsh_capt_161, xsh_match);
fclose(fid);

```

```
%% ============================================================
%  Section E - Plots 8-17: unsteady cases (envelope + probe history)
%  Plot 18: Mach envelope for the moderate case
%% ============================================================
irefine_u = 4;
jmax_u    = 40*irefine_u + 1;

cases = struct( ...
    'name', {'moderate', 'long', 'short', 'small', 'large'}, ...
    'dpvar', {0.10, 0.10, 0.10, 0.02, 0.20}, ...
    'timeper', {1.0, 2.0, 0.5, 1.0, 1.0}, ...
    'slug_env', {'fig08_unsteady_env_mod',   'fig10_unsteady_env_long', ...
                 'fig12_unsteady_env_short', 'fig14_unsteady_env_small', ...
                 'fig16_unsteady_env_large'}, ...
    'slug_time',{'fig09_unsteady_probe_mod',   'fig11_unsteady_probe_long', ...
                 'fig13_unsteady_probe_short', 'fig15_unsteady_probe_small', ...
                 'fig17_unsteady_probe_large'});

R_moderate = [];   % cache for plot 18

for c = 1:numel(cases)
    cs = cases(c);
    opts = struct('gamma', gamma, 'amach0', amach0, 'p0', p0, 'rho0', rho0, ...
                  'xlength', xlength, 'xsh', xsh_init, ...
                  'jmax', jmax_u, 'cfl', 0.9, ...
                  'itsteady', 1500*irefine_u, 'imeth', -1, ...
                  'dpvar', cs.dpvar, 'timeper', cs.timeper, 'iper', 6);

    fprintf('Running unsteady case "%s" (dpvar=%.2f, timeper=%.2f) ...\n', ...
            cs.name, cs.dpvar, cs.timeper);
    R = unsteady_run(opts);

    if strcmp(cs.name, 'moderate'); R_moderate = R; end

    % --- envelope plot ---
    fh = figure('Position', [100 100 800 480]);
    plot(R.x, R.p_steady, 'k-', 'LineWidth', 2.5); hold on;
    n_snap = size(R.p_envelope, 2);
    cm = parula(max(n_snap, 2));
    for k = 1:n_snap
        plot(R.x, R.p_envelope(:,k), '-', 'Color', cm(k,:), 'LineWidth', 1.0);
    end
    hold off; grid on; box on;
    xlabel('x'); ylabel('p');
    title(sprintf(['Unsteady pressure envelope (last period): ' ...
                   '\\Deltap_{var}=%.2f, T=%.2f'], cs.dpvar, cs.timeper));
    legend({'Steady reference', 'Snapshots over last period'}, ...
           'Location', 'NorthWest');
```

```
    axis([0 xlength 0 1.05]);
    save_fig(fh, cs.slug_env);
    close(fh);

    % --- probe time history ---
    fh = figure('Position', [100 100 800 420]);
    plot(R.t_probe, R.p_probe, 'r-', 'LineWidth', 1.5);
    grid on; box on;
    xlabel('Time (fraction of period)');
    ylabel(sprintf('p at x = x_{sh}+0.05 L = %.3f', R.xprobe));
    title(sprintf(['Probe pressure history (6 periods): ' ...
                   '\\Deltap_{var}=%.2f, T=%.2f'], cs.dpvar, cs.timeper));
    save_fig(fh, cs.slug_time);
    close(fh);
end

%% --- Plot 18: Mach envelope for moderate case ---
fh = figure('Position', [100 100 800 480]);
plot(R_moderate.x, R_moderate.amach_steady, 'k-', 'LineWidth', 2.5); hold on;
n_snap = size(R_moderate.amach_envelope, 2);
cm = parula(max(n_snap, 2));
for k = 1:n_snap
    plot(R_moderate.x, R_moderate.amach_envelope(:,k), '-', ...
         'Color', cm(k,:), 'LineWidth', 1.0);
end
plot([0 xlength], [1 1], 'k--', 'LineWidth', 0.75);
hold off; grid on; box on;
xlabel('x'); ylabel('Mach number');
title('Unsteady Mach-number envelope (moderate case: \Deltap_{var}=0.10, T=1)');
legend({'Steady reference', 'Snapshots over last period', 'M = 1'}, ...
       'Location', 'NorthEast');
axis([0 xlength 0 2.5]);
save_fig(fh, 'fig18_unsteady_mach_env_mod');
close(fh);

disp('All 18 figures written to outputs/figures/.');
```

Listing 8.2.2: Parameterized unsteady-flow solver.
`./src/unsteady_run.m`

```
% unsteady_run.m
%
% Parameterized, plot-free version of quasi1d.m. Runs the explicit
% flux-vector-split unsteady solver from a space-marched initial
% condition and returns time-history data for plotting by the driver.
%
% INPUT (struct opts):
%   opts.gamma, opts.amach0, opts.p0, opts.rho0
%   opts.xlength, opts.xsh
```

```
%   opts.jmax, opts.cfl
%   opts.itsteady (iterations to converge to steady before forcing)
%   opts.imeth   (-2 AUSM, -1 vanLeer, +1 Steger-Warming)
%   opts.dpvar   (fractional exit-pressure oscillation)
%   opts.timeper (period scale; itperiod = 400*irefine*timeper)
%   opts.iper    (number of forced periods)
%   opts.itplots (snapshots per period; default 16)
%
% OUTPUT (struct R):
%   R.x                 (1 x jmax)  grid
%   R.area              (1 x jmax)  cell-center area
%   R.p_steady          pressure at end of itsteady ramp-up
%   R.amach_steady      Mach at end of itsteady
%   R.p_envelope        (jmax x Nsnap) snapshots taken during the LAST forced period
%   R.amach_envelope    (jmax x Nsnap) Mach snapshots, same cadence
%   R.t_envelope        (1 x Nsnap) snapshot times in fraction-of-period
%   R.t_probe           (1 x Nfull) probe-time vector (fraction of period) over all forced
    periods
%   R.p_probe           (1 x Nfull) probe pressure history
%   R.normit, R.normres  residual histories (1 x itmax)
%   R.xsh, R.xprobe     bookkeeping
%   R.p_final           final pressure profile
%
function R = unsteady_run(opts)
    gamma   = opts.gamma;
    amach0  = opts.amach0;
    p0      = opts.p0;
    rho0    = opts.rho0;
    xlength = opts.xlength;
    xsh     = opts.xsh;
    jmax    = opts.jmax;
    cfl     = opts.cfl;
    itsteady= opts.itsteady;
    imeth   = opts.imeth;
    dpvar   = opts.dpvar;
    timeper = opts.timeper;
    iper    = opts.iper;
    if isfield(opts,'itplots'); itplots = opts.itplots; else; itplots = 16; end

    % derive irefine from jmax (jmax = 40*irefine + 1)
    irefine = (jmax - 1) / 40;
    itperiod = round(irefine * 400 * timeper);
    itmax = itsteady + itperiod * iper;

    dx = xlength / (jmax - 1);
    x = 0:dx:xlength;
    area    = calcarea(x);
    areaint = calcarea(x + 0.5*dx);
```

```
57
58      xprobe = xsh + 0.05 * xlength;
59      iloc = floor(xprobe/xlength*(jmax-1) + 1);
60      fprobe = (xprobe - x(iloc)) / dx;
61
62      [rho_sp, u_sp, p_sp, e_sp, amach_sp] = ...
63          spacemarch(gamma, amach0, p0, rho0, xsh, x, area, 1);
64      rho = rho_sp; u = u_sp; p = p_sp; e = e_sp; amach = amach_sp;
65      pend = p(jmax);
66
67      q = loadq(rho, u, e, area);
68
69      dq = zeros(3, jmax);
70      fluxjp = zeros(3, jmax-1);
71      normit  = zeros(1, itmax);
72      normres = zeros(1, itmax);
73
74      n_probe = itperiod * iper;
75      p_probe = zeros(1, n_probe);
76
77      snap_stride = max(1, round(itperiod / itplots));
78      n_snap_max = ceil(itperiod / snap_stride) + 2;
79      p_envelope     = zeros(jmax, n_snap_max);
80      amach_envelope = zeros(jmax, n_snap_max);
81      t_envelope     = zeros(1, n_snap_max);
82      n_snap = 0;
83
84      p_steady = p_sp;
85      amach_steady = amach_sp;
86
87      for iter = 1:itmax
88          if iter < itsteady
89              dtdx = min(cfl ./ (abs(u) + u ./ amach));
90          end
91
92          if imeth == 1
93              [fluxp, fluxn] = steger_flux(gamma, area, rho, u, p, e);
94              fluxjp(:, 1:jmax-1) = fluxp(:, 1:jmax-1) + fluxn(:, 2:jmax);
95          elseif imeth == -1
96              [fluxp, fluxn] = vanl_flux(gamma, area, rho, u, p, e);
97              fluxjp(:, 1:jmax-1) = fluxp(:, 1:jmax-1) + fluxn(:, 2:jmax);
98          elseif imeth == -2
99              fluxjp = ausm_flux(gamma, areaint, rho, u, p, e);
100         end
101
102         if imeth ≤ 1
103             dq(:, 2:jmax-1) = -dtdx * (fluxjp(:, 2:jmax-1) - fluxjp(:, 1:jmax-2));
104             dq(2, 2:jmax-1) = dq(2, 2:jmax-1) + ...
```

```
                dtdx * p(2:jmax-1) .* (areaint(2:jmax-1) - areaint(1:jmax-2));
        end

        dq = calcbc(gamma, dq, p, rho, u, area, dtdx, dpvar, pend, ...
                    itperiod, iter, itsteady);

        rsq = sqrt(dq(1,:).^2 + dq(2,:).^2 + dq(3,:).^3);
        normres(iter) = norm(rsq, 2);
        normit(iter)  = normres(iter);

        q = q + dq;
        [rho, u, p, e, amach] = loadpr(q, area, gamma);

        if iter == itsteady
            pend = p(jmax);
            p_steady = p;
            amach_steady = amach;
        end

        if iter > itsteady
            k = iter - itsteady;
            p_probe(k) = (1 - fprobe) * p(iloc) + fprobe * p(iloc + 1);

            % snapshot during LAST forced period
            if iter > itmax - itperiod && mod(iter - itsteady, snap_stride) == 0
                n_snap = n_snap + 1;
                p_envelope(:, n_snap)     = p(:);
                amach_envelope(:, n_snap) = amach(:);
                t_envelope(n_snap) = (iter - itsteady) / itperiod;
            end
        end
    end

    p_envelope     = p_envelope(:, 1:n_snap);
    amach_envelope = amach_envelope(:, 1:n_snap);
    t_envelope     = t_envelope(1:n_snap);

    R = struct();
    R.x = x;
    R.area = area;
    R.p_steady = p_steady;
    R.amach_steady = amach_steady;
    R.p_envelope = p_envelope;
    R.amach_envelope = amach_envelope;
    R.t_envelope = t_envelope;
    R.t_probe = (1:n_probe) / itperiod;
    R.p_probe = p_probe;
    R.normit = normit;
```

```
153     R.normres = normres;
154     R.xsh = xsh;
155     R.xprobe = xprobe;
156     R.p_final = p;
157     R.amach_final = amach;
158     R.itsteady = itsteady;
159     R.itperiod = itperiod;
160 end
```